

NEUROSCOPE

Brain Tumor MRI Classification System

Pipeline, Model Architecture, and Application Report

Dataset: PMRAM — Bangladeshi Brain Cancer MRI Dataset

Model: Swin-Tiny backbone, JEPA-style self-supervised pretraining + fine-tuned classifier

Test accuracy: 98.45% across four classes (Glioma, Meningioma, Pituitary, No Tumor)

Application: Flask backend + Three.js 3D anatomical reference viewer

Research / educational project. Not a diagnostic device.

1. Project Overview

NeuroScope is an end-to-end system that classifies brain MRI slices into four categories — **Glioma**, **Meningioma**, **Pituitary tumor**, and **No Tumor** — using a Swin-Transformer-based deep learning model, and serves predictions through a web application that displays a Grad-CAM style explanation on the scan itself, alongside a 3D anatomical reference visualization.

The system is built in three stages:

- **Data preparation** — loading and splitting the labeled MRI dataset.
- **Model training** — a two-phase pipeline: self-supervised JEPA-style pretraining of the visual backbone, followed by supervised fine-tuning of a classification head.
- **Application layer** — a Flask backend serving a browser-based interface where a user uploads a scan and receives a classification, a heatmap overlay, and a 3D visual reference.

2. Dataset

Source: PMRAM — Bangladeshi Brain Cancer MRI Dataset (Mendeley Data, CC BY 4.0), collected from three hospitals: Ibn Sina Medical College Dhaka, Dhaka Medical College & Hospital, and Cumilla Medical College.

Class	Raw images	Augmented images
Glioma	400	1,500
Meningioma	400	1,500
Pituitary	400	1,500
No Tumor	400	1,500
Total	1,600	6,000

All images are resized to 512×512px. The augmented set was produced with rotation ($\pm 45^\circ$), width/height shift ($\pm 20\%$), shear (20%), zoom ($\pm 20\%$), and horizontal flip.

Split used for this project (stratified by class):

Split	Images
Train	4,202
Validation	901
Test	901

3. Model Architecture

3.1 Visual Backbone

Swin-Tiny (timm: swin_tiny_patch4_window7_224) is used as the visual backbone. For a 224×224 input, it produces a **7×7 grid of patch tokens** (49 tokens), each of dimension **768**.

3.2 Stage 1 — JEPA-style Self-Supervised Pretraining

Implemented in `model.py`: `JEPAModel`. Four components work together:

Component	Description
Context encoder	The online Swin-Tiny backbone (trainable)
Target encoder	EMA (momentum) copy of the context encoder, decay = 0.999, no gradient
Predictor	8-layer causal Transformer predicting masked patch representations from visible patches
Decoder	2-layer cross-attention module for spatial completion of predicted tokens

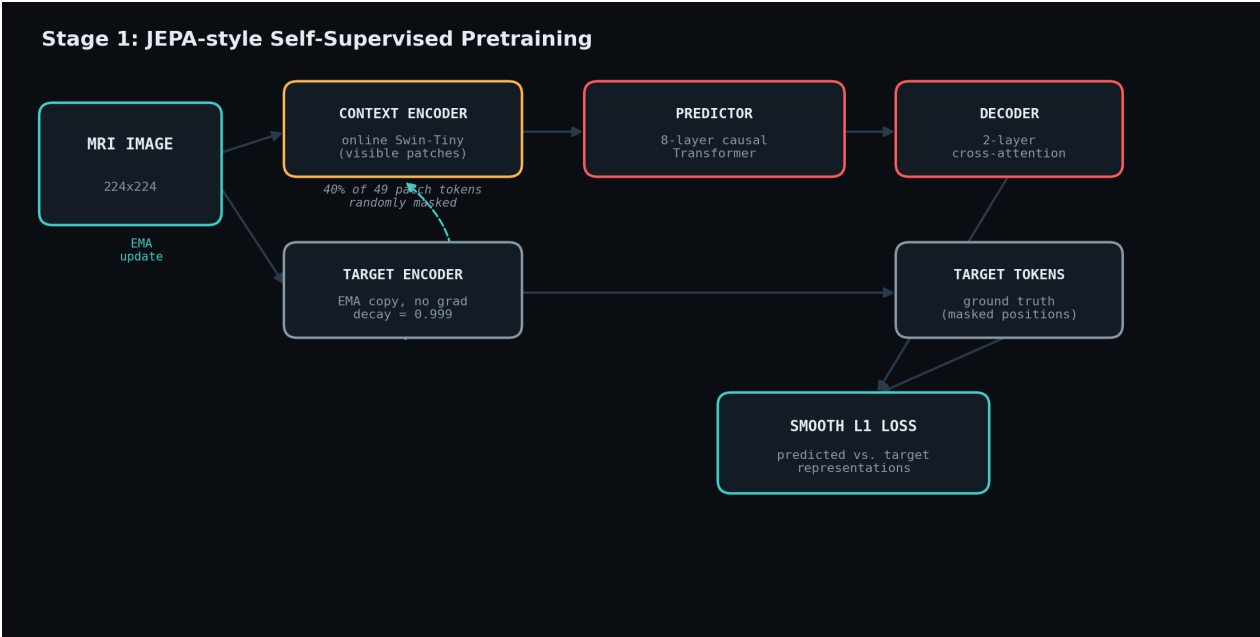


Figure 1 — JEPA-style self-supervised pretraining architecture

Procedure per training step:

- 1 Randomly mask 40% of the 49 patch tokens (`mask_ratio = 0.4`).
- 2 The context encoder processes only the visible (unmasked) tokens.
- 3 The target encoder (EMA, no gradient) processes the full image to produce ground-truth representations for the masked positions.
- 4 The predictor, given context tokens + learnable mask tokens + positional embeddings, predicts the masked tokens' representations.

- 5 The decoder refines predictions via cross-attention against context tokens.
- 6 Loss: **smooth L1** between predicted and target representations at the masked positions.
- 7 The target encoder is updated as an EMA of the context encoder's weights after every step.

3.3 Stage 2 — Supervised Fine-Tuning

Implemented in `model.py`: `MRIClassifier`. The context encoder from Stage 1 is reused; a lightweight head (LayerNorm → Linear 768→4) is added on top of the mean-pooled patch tokens, and the whole model is fine-tuned with standard cross-entropy loss against the four class labels.

3.4 Hyperparameters (both stages)

Parameter	Value
Optimizer	AdamW, weight decay 0.05
Learning rate	3×10^{-4}
Batch size	32
Epochs	15 (pretraining) + 15 (fine-tuning)
Image size	224 x 224

4. Training Pipeline

Code structure:

```
dataset.py - loads images from class-named folders, builds train/val/test splits, defines transforms
model.py - SwinBackbone, JEPAModel (predictor/decoder/EMA), MRIClassifier
train.py - orchestrates both training stages, saves checkpoints + plots
predict.py - CLI inference tool (single image / folder / labeled eval)
```

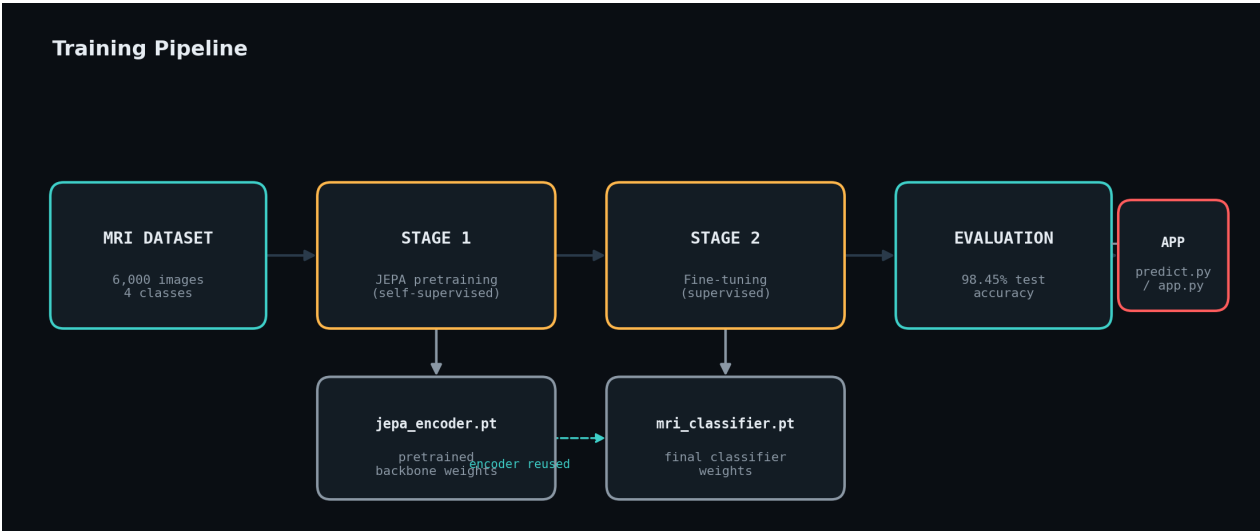


Figure 2 — End-to-end training pipeline

Outputs written to outputs/: jepa_pretrain_loss.png, training_curves.png, confusion_matrix.png, classification_report.txt, history.json, jepa_encoder.pt, mri_classifier.pt.

5. Evaluation Results

Test set accuracy: 98.45% (901 held-out images)

Class	Precision	Recall	F1-score	Support
Glioma	0.9825	0.9912	0.9868	226
Meningioma	0.9911	0.9911	0.9911	225
Pituitary	0.9657	1.0000	0.9825	225
No Tumor	1.0000	0.9556	0.9773	225
Accuracy			0.9845	901
Macro avg	0.9848	0.9845	0.9844	901
Weighted avg	0.9848	0.9845	0.9844	901

All four classes exceed 0.97 F1-score. Pituitary shows perfect recall (no missed cases) but slightly lower precision (a few other classes misclassified as Pituitary); No Tumor shows perfect precision but slightly lower recall (a few normal scans misclassified as a tumor class). Standalone inference on out-of-sample test images (predict.py) confirmed correct classification with 99.9–100% confidence across all four classes.

6. Inference / Testing Tool (predict.py)

A command-line tool supporting three modes against the saved model checkpoint:

- `--image <path>` — classify a single image, print per-class confidence.
- `--image_dir <folder>` — batch-classify a folder of unlabeled images, saving predictions.csv.
- `--data_dir <folder>` — evaluate against a labeled folder, producing a fresh accuracy report and confusion matrix.

7. Web Application

7.1 Architecture

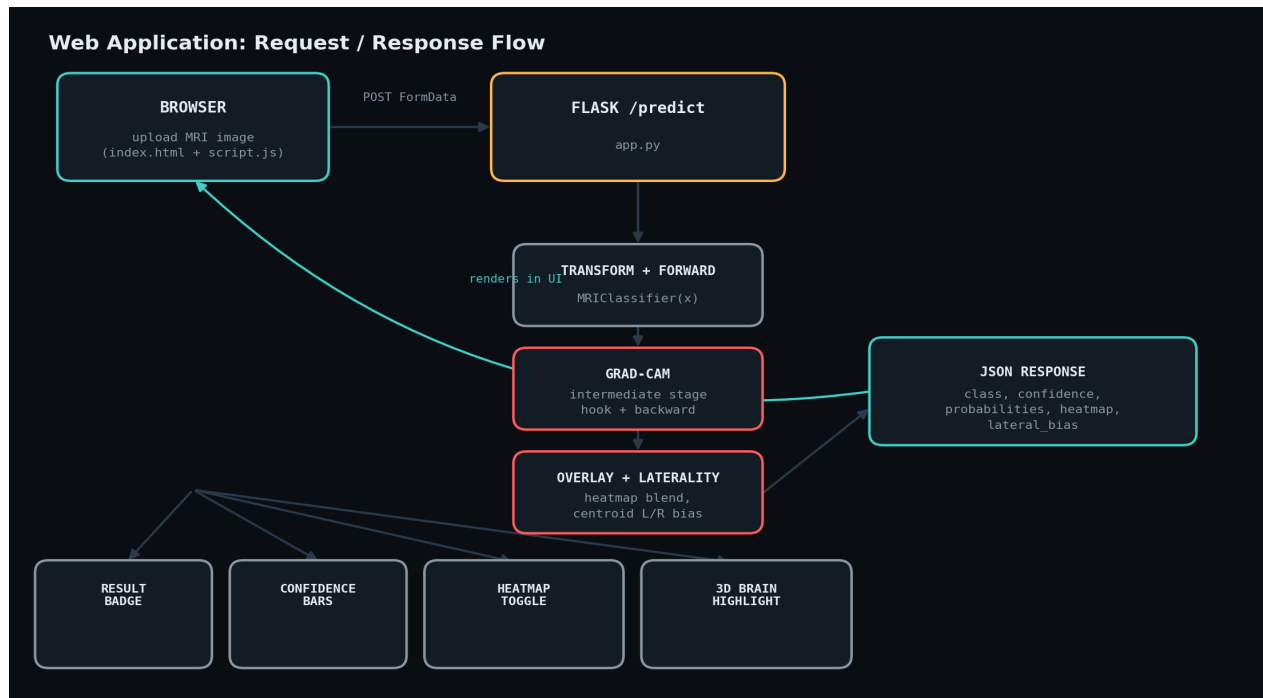


Figure 3 — Web application request/response flow

7.2 Backend (app.py)

- Loads the trained MRIClassifier once at first request (lazy singleton).
- `/predict` endpoint accepts an uploaded image, runs inference, and computes a Grad-CAM-style relevance map:
 - Hooks into an intermediate Swin stage (a finer $\sim 14 \times 14$ token grid, rather than the coarse final 7×7 grid) for sharper localization, with a fallback to the final stage if that hook can't attach.
 - Uses the classic Grad-CAM formula: channel-importance weights (gradient averaged over spatial positions) applied to the activation map, followed by ReLU.
 - The relevance map is blended onto the original image with per-pixel alpha proportional to relevance, so only genuinely high-relevance regions are tinted.
 - A left/right laterality bias is computed from the heatmap's horizontal centroid, used to pick which side's 3D marker to light up.

7.3 Frontend (templates/index.html, static/style.css, static/script.js)

A radiology-console-styled interface with three panels: **scan input** (upload, scanning animation, heatmap toggle), **classification** (predicted class, confidence bars), and **anatomical reference** — a Three.js 3D viewer loading a user-provided STL brain model, with glow markers indicating the region typically associated with the predicted class:

Predicted class	3D marker behavior
Glioma	Left/right cerebral hemisphere (gliomas arise broadly in glial tissue)
Meningioma	Three surface patches: skull vertex, left/right lateral convexity
Pituitary	Single marker at the base/center of the brain
No Tumor	No highlight

Left/right selection for Glioma and Meningioma markers is informed by the lateral_bias value from the 2D heatmap, so the same predicted class doesn't always light up symmetrically regardless of the actual scan.

7.4 Scope Limitation (by design)

*The 3D highlight is a **general anatomical association for the predicted class**, not a true patient-specific 3D localization — the model is a single-2D-slice classifier, not a segmentation or 3D-localization model, so it has no way to know true depth/position within the skull. Only left/right laterality is derived from the actual image; everything else about the 3D marker position is illustrative. This limitation is stated directly in the app's footer disclaimer. The 2D heatmap, by contrast, is a genuine (if coarse) explanation of what drove the prediction for that specific image.*

8. Project File Structure

```
project B/  
  dataset.py      – Data loading, splitting, transforms  
  model.py       – SwinBackbone, JEPAModel, MRIClassifier  
  train.py       – Training pipeline (pretrain + fine-tune)  
  predict.py     – CLI inference / evaluation tool  
  app.py        – Flask backend (web app)  
  requirements.txt  
  outputs/      – checkpoints, plots, reports  
  templates/index.html  
  static/style.css, script.js, models/brain.stl
```

9. Known Limitations

- **Not a diagnostic device.** Built and evaluated for research/educational purposes only.
- **Single-slice input.** The model classifies one 2D MRI slice; true spatial localization within the brain is out of scope.
- **Schematic 3D brain.** The uploaded STL has no annotated anatomical sub-regions, so markers use fixed anatomical-fraction offsets, not real landmarks in the mesh.
- **Dataset scale.** Originates from only 400 raw images per class before augmentation, from three hospitals in one country — generalization to other scanners/populations is untested.
- **Grad-CAM is an approximation.** It highlights regions that influenced the decision, not a verified tumor boundary or segmentation.

10. Possible Future Work

- Train on full 3D scan volumes (or multi-slice series) for genuine 3D tumor localization.
- Fine-tune on a larger, multi-institution dataset to test generalization.
- Add true tumor segmentation (pixel/voxel-level masks) if annotated data becomes available.
- Replace the ImageNet-pretrained Swin-Tiny backbone with an actual medical-domain pretrained checkpoint if one becomes available.